

SDC 학생강사

컴퓨터공학 입문자를 위한 자료구조론

5주차

CONTENTS

01

지난 시간 문제 분석

해당 목차에 대한 설명을 입력해 주세요

02

우선순위 큐

해당 목차에 대한 설명을 입력해 주세요

03

힙

해당 목차에 대한 설명을 입력해 주세요

01

지난 시간 문제 분석

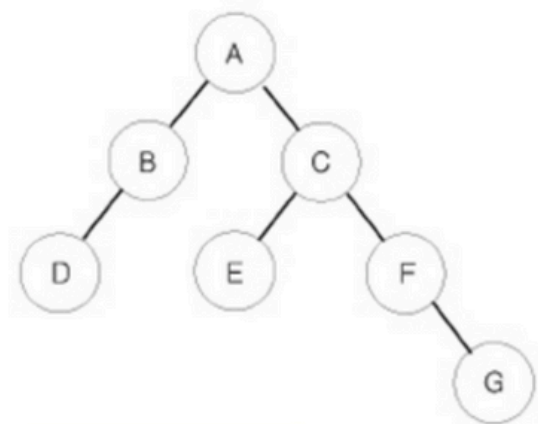
문제 1

트리 순회

시간 제한	메모리 제한	제출	정답	맞힌 사람	정답 비율
2 초	128 MB	77478	50903	39241	67.473%

문제

이진 트리를 입력받아 전위 순회(preorder traversal), 중위 순회(inorder traversal), 후위 순회(postorder traversal)한 결과를 출력하는 프로그램을 작성하시오.



예를 들어 위와 같은 이진 트리가 입력되면,

- 전위 순회한 결과 : ABDCEFG // (루트) (왼쪽 자식) (오른쪽 자식)
- 중위 순회한 결과 : DBAECFG // (왼쪽 자식) (루트) (오른쪽 자식)
- 후위 순회한 결과 : DBEGFCA // (왼쪽 자식) (오른쪽 자식) (루트)

01

지난 시간 문제 분석

문제 1

```
1 class Node
2 {
3 public:
4     char data;
5     Node *parent;
6     Node *left;
7     Node *right;
8
9     Node(char data)
10    {
11        this->data = data;
12        parent = nullptr;
13        left = nullptr;
14        right = nullptr;
15    }
16 };
```

```
1 class Tree
2 {
3 private:
4     vector<Node*> nodes;
5     Node *root;
6
7 public:
8     Tree()
9     {
10         this->root = nullptr;
11     }
12
13     Node *find(char data)
14     {
15         for (Node *n : nodes)
16         {
17             if (n->data == data)
18             {
19                 return n;
20             }
21         }
22         return nullptr;
23     }
24     Node *getRoot()
25     {
26         return root;
27     }
```

01

지난 시간 문제 분석

문제 1

```
1 void insert(char data)
2 {
3     Node *node = new Node(data);
4     if (root == nullptr)
5     {
6         this->root = node;
7     }
8     nodes.push_back(node);
9 }
10
11 void setLeft(char p, char c)
12 {
13     Node *node = this->find(p);
14     Node *child = this->find(c);
15     node->left = child;
16 }
17 void setRight(char p, char c)
18 {
19     Node *node = this->find(p);
20     Node *child = this->find(c);
21     node->right = child;
22 }
```

```
1
2 void preorder(Node *node)
3 {
4     if (!node)
5         return;
6     cout << node->data;
7     preorder(node->left);
8     preorder(node->right);
9 }
10
11 void inorder(Node *node)
12 {
13     if (!node)
14         return;
15     inorder(node->left);
16     cout << node->data;
17     inorder(node->right);
18 }
19
20 void postorder(Node *node)
21 {
22     if (!node)
23         return;
24     postorder(node->left);
25     postorder(node->right);
26     cout << node->data;
27 }
```

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5
6     int N;
7     cin >> N;
8     Tree *tree = new Tree();
9     while (N--)
10     {
11         char p, l, r;
12         cin >> p >> l >> r;
13         tree->insert(p);
14
15         if (l != '.')
16         {
17             if (tree->find(l) == nullptr)
18                 tree->insert(l);
19             tree->setLeft(p, l);
20         }
21         if (r != '.')
22         {
23             if (tree->find(r) == nullptr)
24                 tree->insert(r);
25             tree->setRight(p, r);
26         }
27     }
28
29     Node *root = tree->getRoot();
30
31     preorder(root);
32     cout << endl;
33     inorder(root);
34     cout << endl;
35     postorder(root);
36
37     return 0;
38 }
```

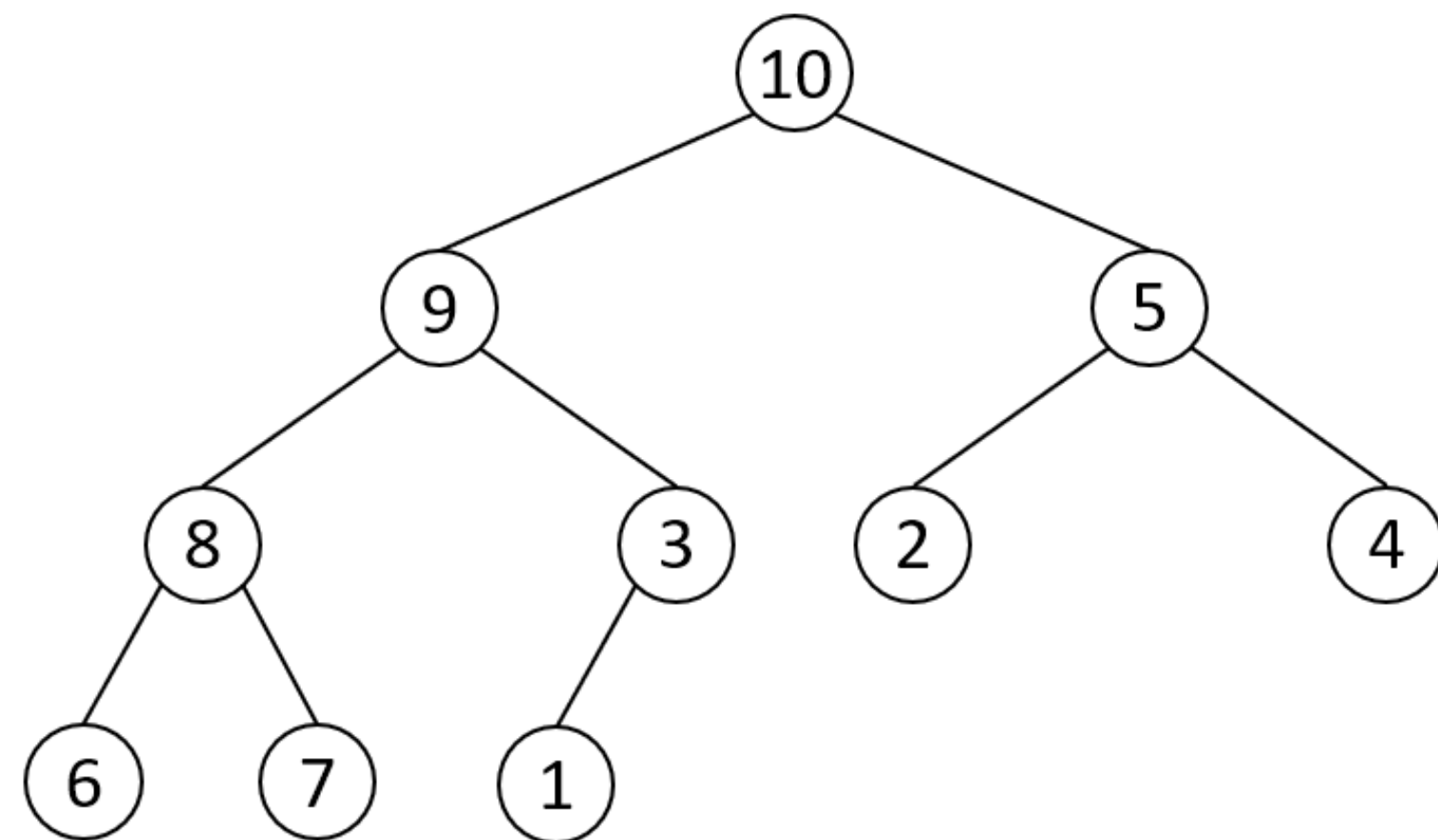

02

힙

힙

힙은 완전 이진 트리(Complete Binary Tree)를 기본으로 하는 자료구조이다.

부모의 값은 항상 자식(들)의 값보다 크거나(최대 힙[max heap]), 작아야(최소 힙[min heap])한다.



02

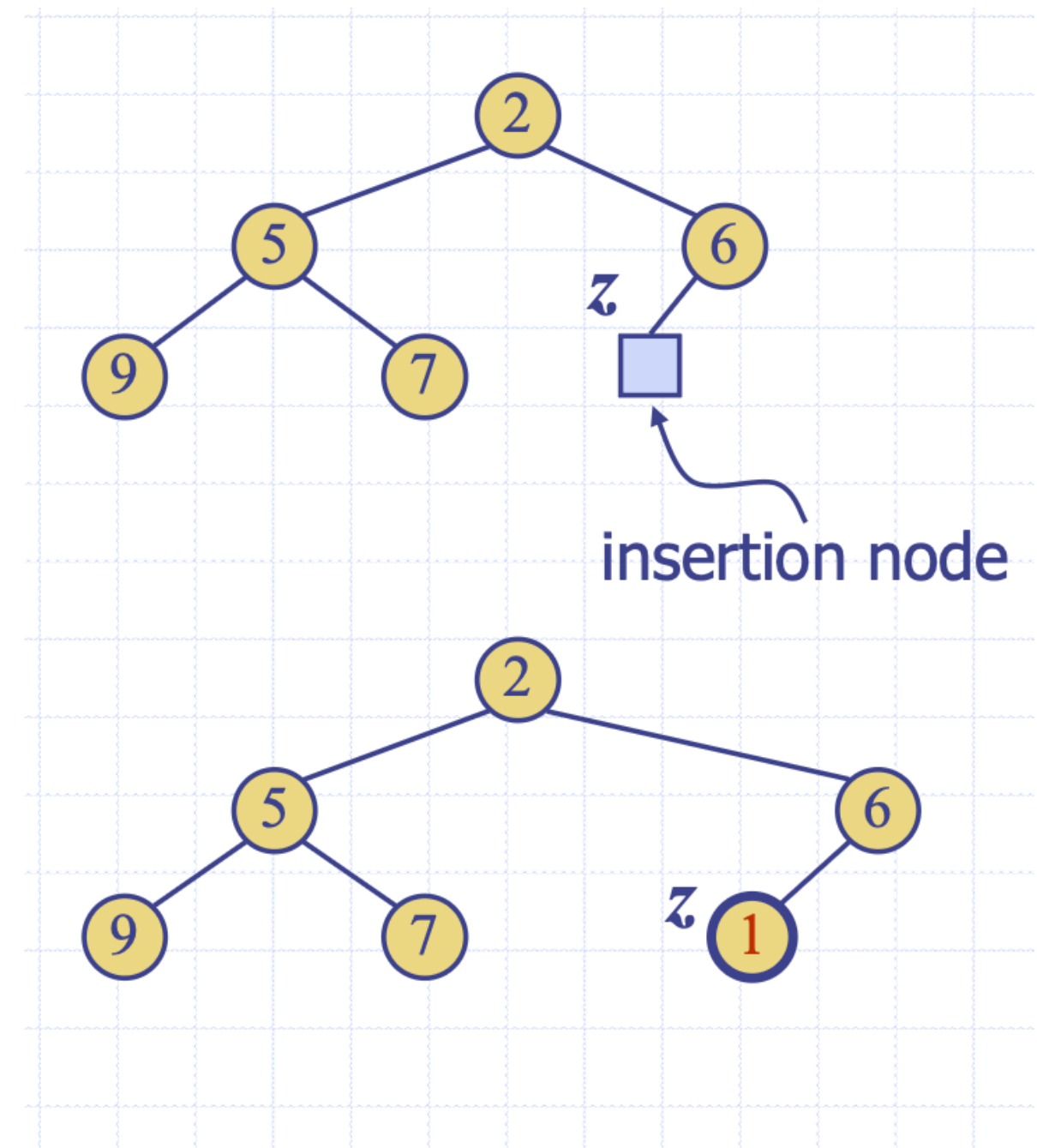
힙

Insert

힙은 Complete Binary Tree 조건을 만족해야 하기에 가장 끝 자리에만 값을 삽입 할 수 있다.

값을 삽입 한 이후에 힙 조건을 만족해야 하기에 구조를 재정렬 할 필요가 있다.

1. 가장 끝 자리에 node를 삽입한다.
2. 힙조건을 만족 할 때 까지 재정렬 한다.



02

힙

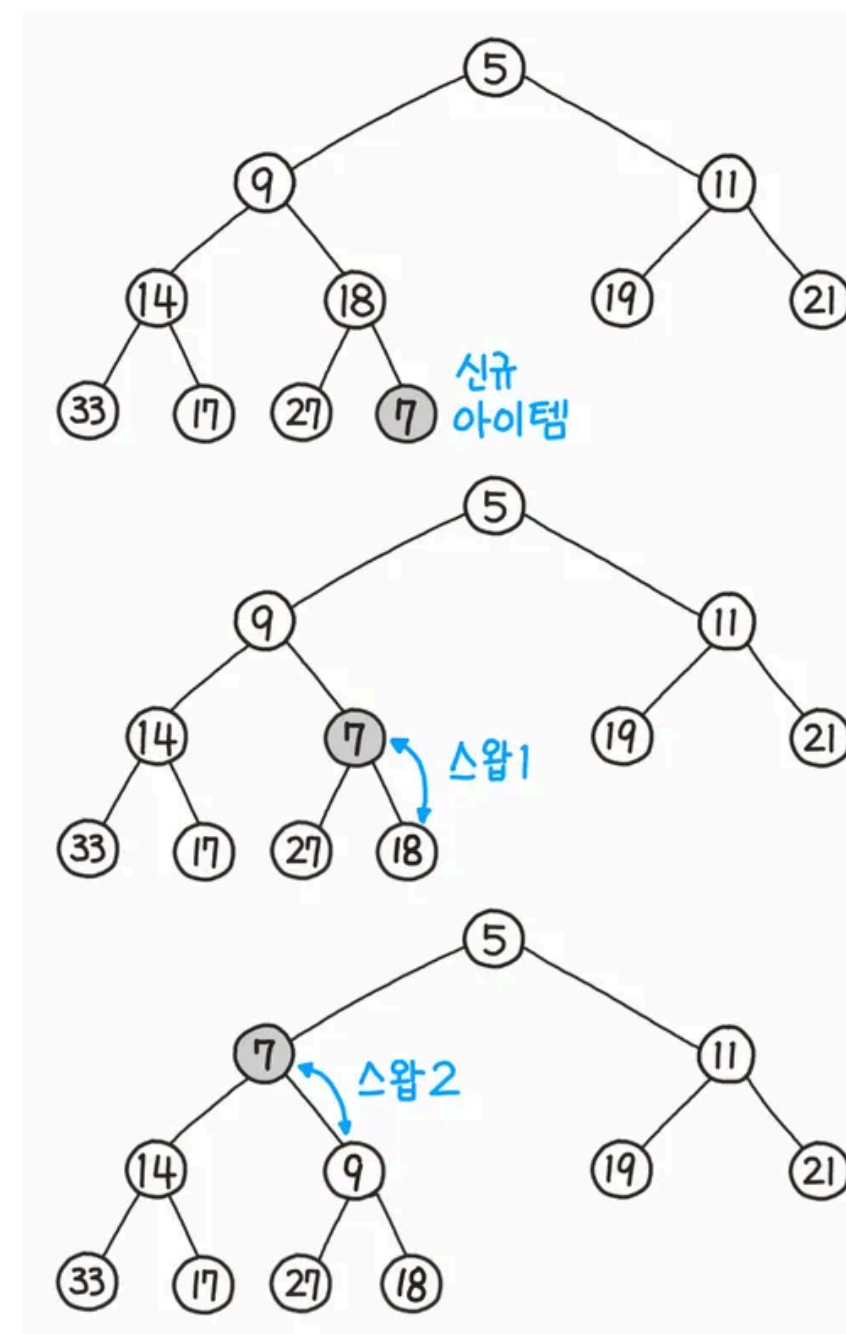
UpHeap

값을 삽입 한 이후 힙 조건이 위배될 가능성이 있다.

Upheap 연산은 새로 들어온 Node와 부모를 비교하며 자리를 바꾸는 연산을 반복한다.

더 이상 올라갈 곳이 없다면 그곳이 새로 삽입된 Node의 위치가 되고,
힙 조건을 만족한다.

최대 swap연산은 힙의 높이만큼 이루어지므로 삽입은 $O(\log n)$ 시간에 수행된다,



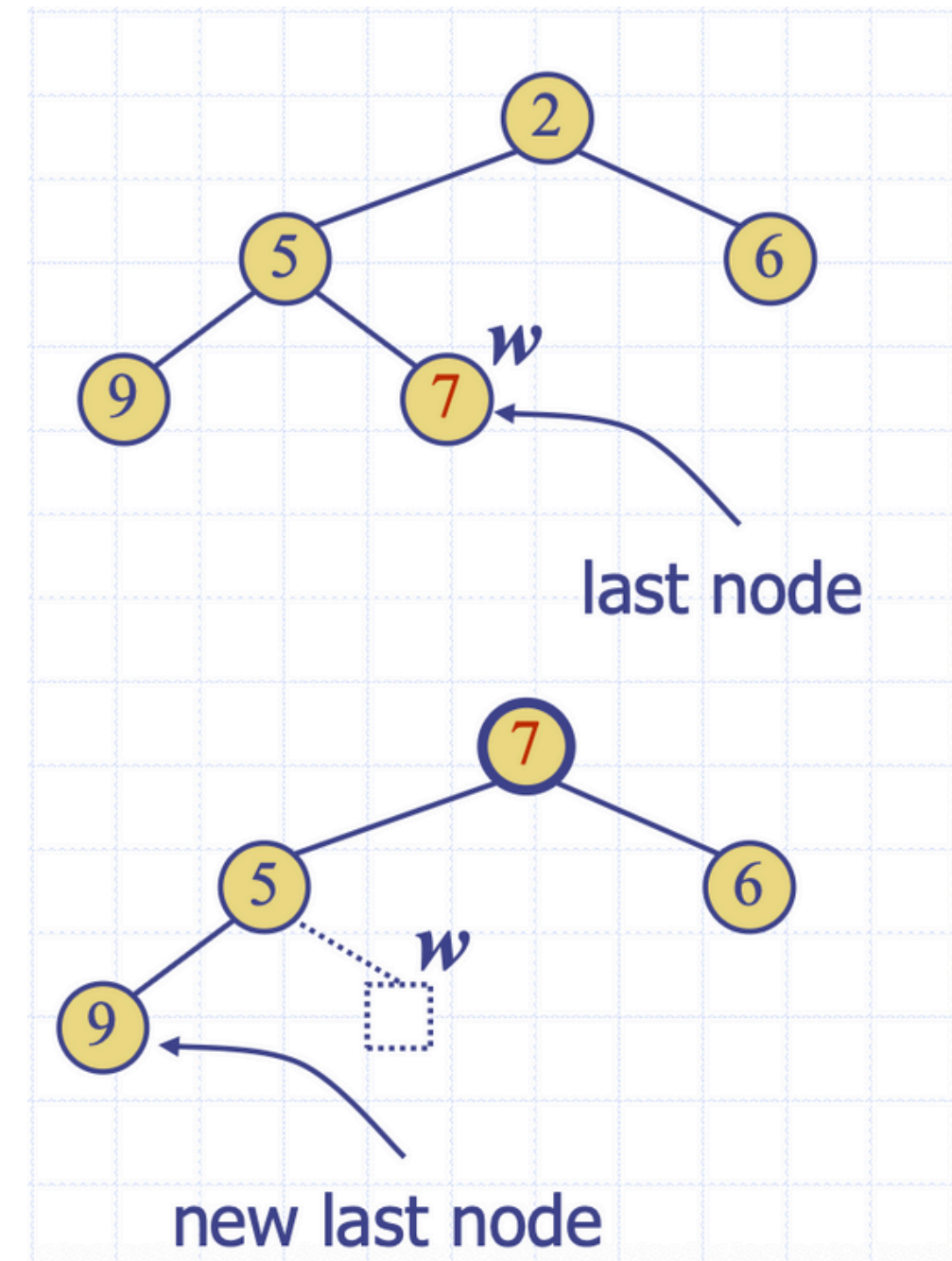
02

힙

Removal

힙에서는 최대값(max), 최소값(min)을 제거 할 수 있다.

1. 루트 노드(최대 혹은 최소값) 을 제거한다.
2. 가장 마지막 node를 루트자리로 이동시킨다.
3. 힙조건을 만족하기 위해 재정렬 한다.



02

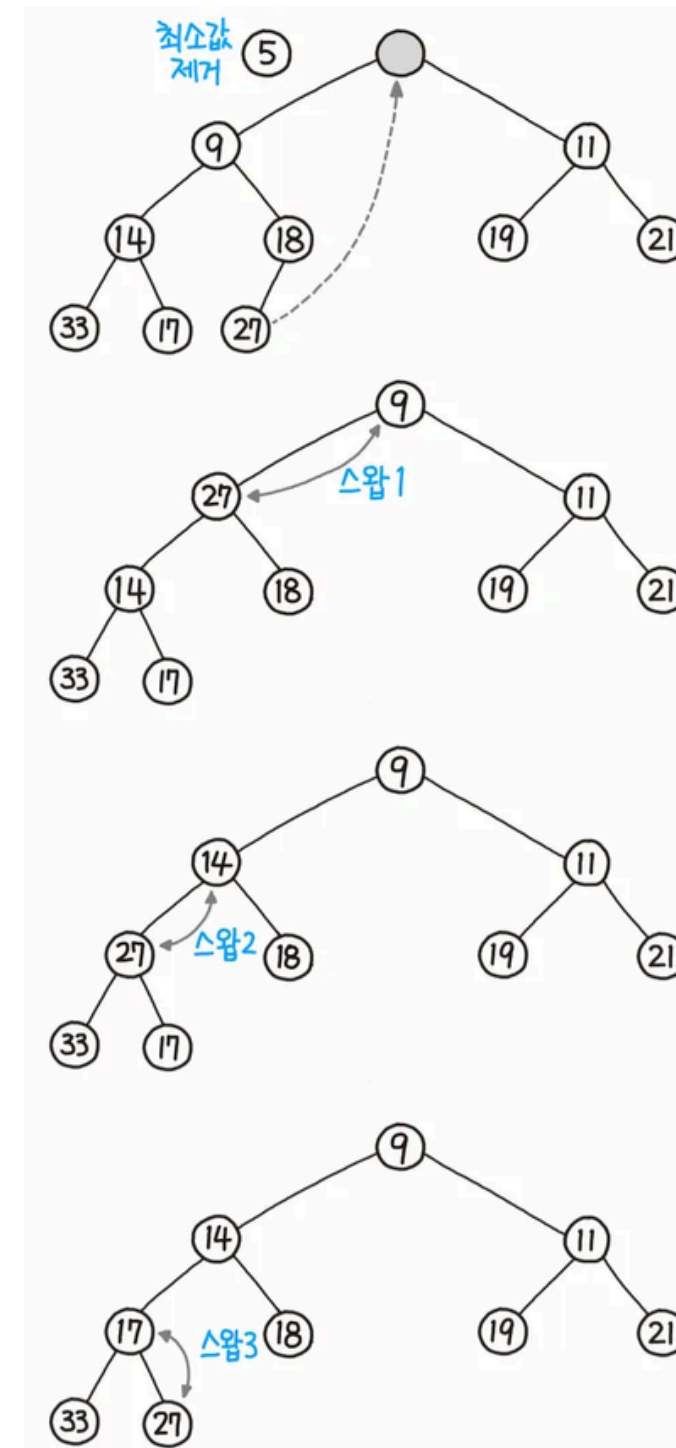
힙

DownHeap

Upheap 연산과 반대로 아래로 내려가며 값을 비교한다.

루트에서 아래로 하나씩 값을 비교하며 swap이 발생하지 않을 때 까지 반복한다.

최대 swap연산은 힙의 높이만큼 이루어지므로 삽입은 $O(\log n)$ 시간에 수행된다,



02

힙

Build Heap

무작위로 정렬된 배열을 바탕으로 힙을 생성해보자

Upheap, downheap 방식을 활용해서 힙을 새로 생성 할 수 있다.

3	5	1	2
----------	----------	----------	----------

02

힙

Build Heap

Upheap 을 활용해서 heap 생성

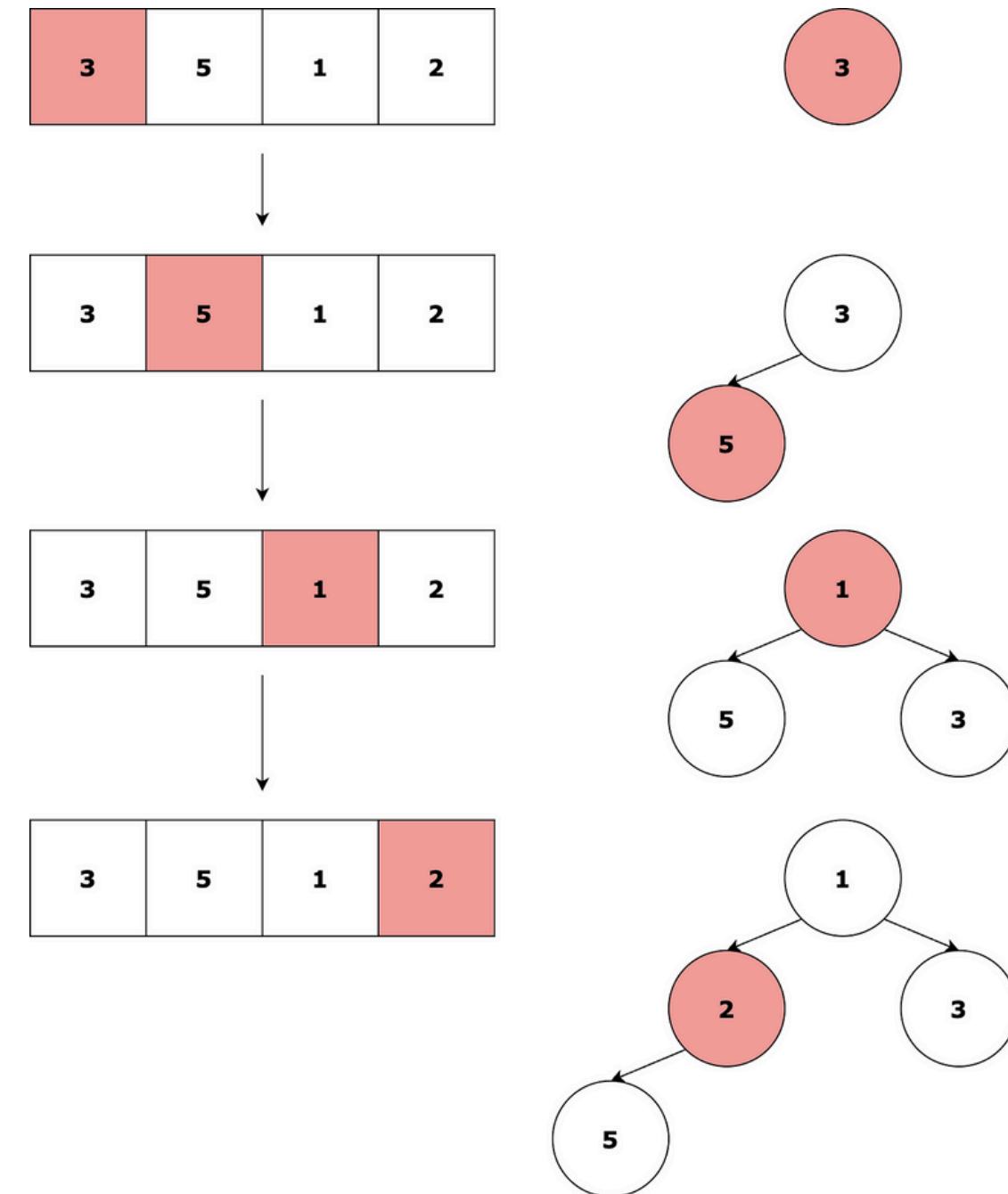
빈 힙에서 원소를 하나씩 삽입하며 Upheap을 수행한다.

매번 삽입이후 heap 조건을 만족하는지 확인하고 up 해야한다.

최악의 경우 매번 swap이 발생한다.

$$O(\log 1 + \dots + \log N) = O(\log(N!)) = O(N \log N)$$

스털링근사



02

힙

Build Heap

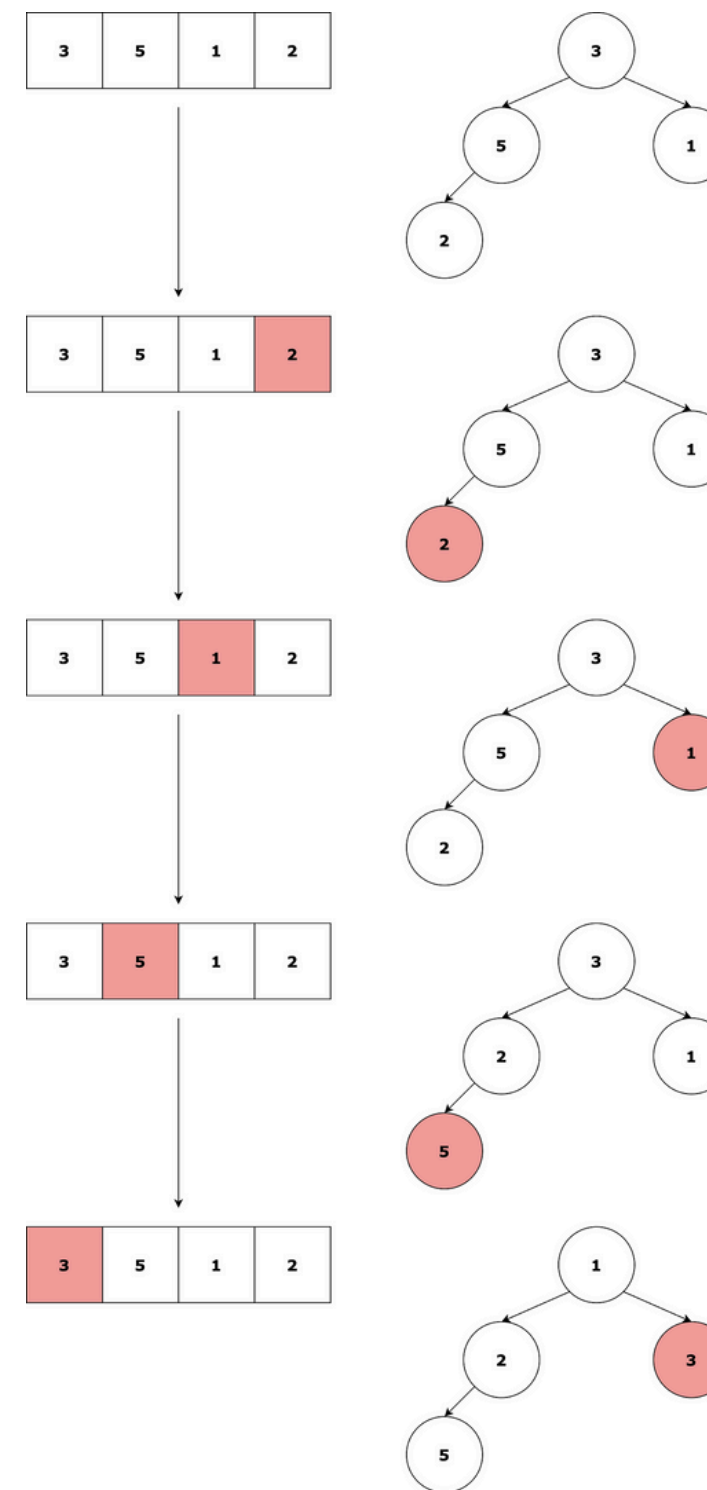
Downheap 을 활용해서 heap 생성

배열을 기준으로 Complete Binary Tree를 생성하고
마지막 Node부터 DownHeap을 수행한다.

리프노드에서 루트로 올라갈수록
아래 서브트리는 정렬된 상태일것이다.

따라서 루트노드로 올라갈수록 연산의 최대 실행 횟
수가 적어질것이다,

높이 H에 대해 $O(0 * (N/2) + (1 * (N/4)) + \dots + (H * 1))$ 이므로 $O(N)$



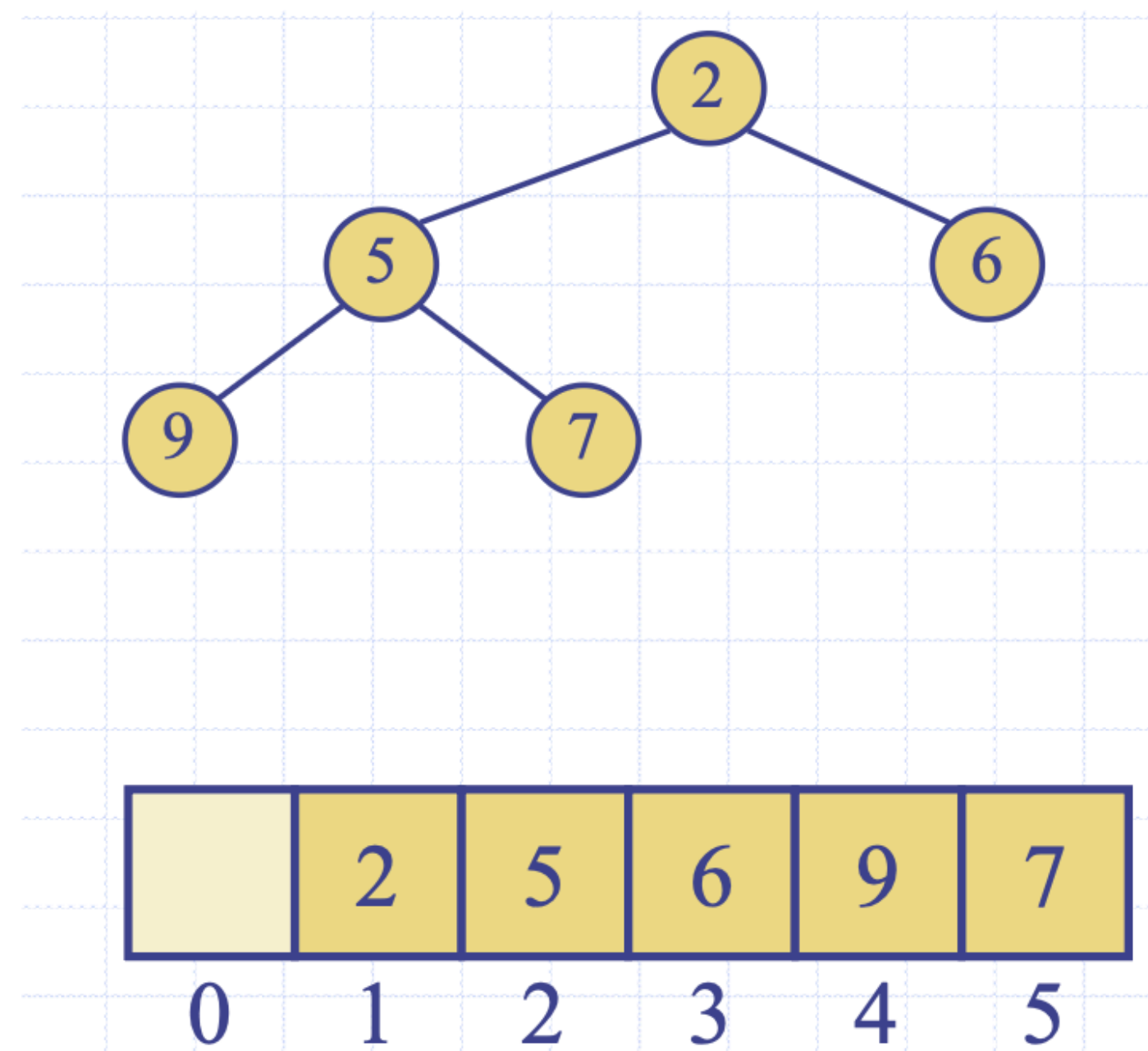
02

힙

vector를 활용한 구현

N개의 Node를 저장하기위해 N+1 크기의 배열로 힙을 구현할 수 있다.

node의 index i 에 대해
left child 의 index = $i*2$
right child 의 index = $i*2+1$
을 만족한다.



02

합

vector를 활용한 구현

```
1  int main()
2  {
3      MinHeap h;
4
5      // 삽입 테스트
6      int data[] = {15, 10, 20, 5, 12, 25, 3};
7      for (int v : data)
8      {
9          cout << "insert(" << v << ")" << endl;
10         h.insert(v);
11     }
12
13     cout << "\n=== Heap 상태 ===" << endl;
14     h.print();
15
16     // 삭제 테스트 (최솟값부터 순서대로 제거)
17     cout << "\n=== removal 테스트 ===" << endl;
18     for (int i = 0; i < 3; i++)
19     {
20         int removed = h.removal();
21         cout << "removed: " << removed << endl;
22     }
23
24     cout << "\n=== 삭제 후 Heap 상태 ===" << endl;
25     h.print();
26
27     return 0;
28 }
29
```

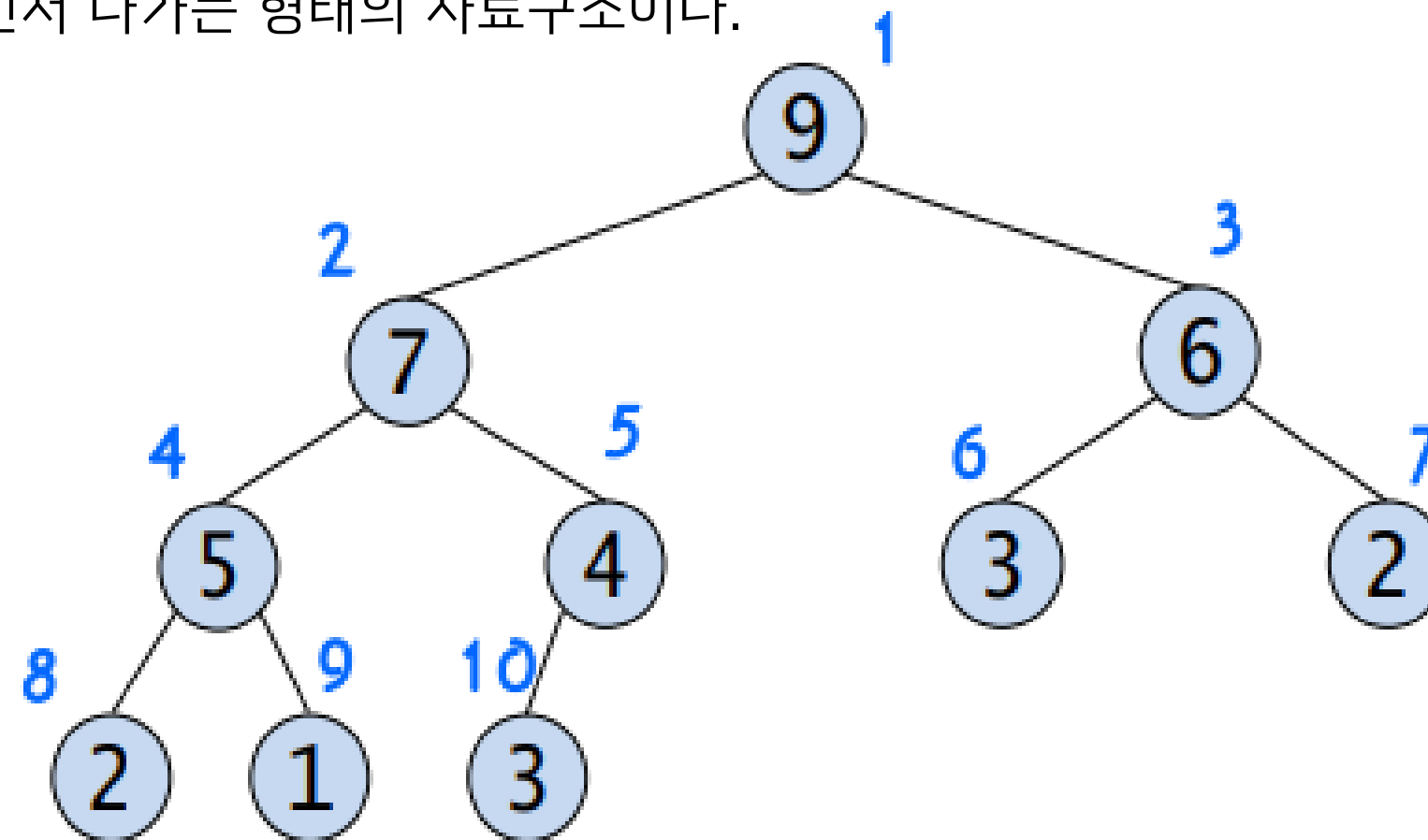
03

우선순위 큐

우선순위 큐

큐(Queue)는 먼저 들어오는 데이터가 먼저 나가는 FIFO(First In First Out) 형식의 자료구조이다.

우선순위 큐(Priority Queue)는 먼저 들어오는 데이터가 아니라,
우선순위가 높은 데이터가 먼저 나가는 형태의 자료구조이다.



0	
1	9
2	7
3	6
4	5
5	4
6	3
7	2
8	2
9	1
10	3
11	

Priority Queue ADT

- A priority queue stores a collection of entries
- Typically, an **entry** is a pair (key, value), where the key indicates the priority
- Main methods of the Priority Queue ADT
 - **insert(e)**
inserts an entry e
 - **removeMin()**
removes the entry with smallest key
- Additional methods
 - **min()**
returns, but does not remove, an entry with smallest key
 - **size(), empty()**
- Applications:
 - Standby flyers
 - Auctions
 - Stock market

03

우선순위 큐

우선순위 큐와 힙

우선순위큐는 삽입,삭제 순서에 상관없이 항상 정렬된 상태를 유지해야한다.

- Implementation with an unsorted list



- Performance:

- **insert** takes $O(1)$ time since we can insert the item at the beginning or end of the sequence
- **removeMin** and **min** take $O(n)$ time since we have to traverse the entire sequence to find the smallest key

- Implementation with a sorted list



- Performance:

- **insert** takes $O(n)$ time since we have to find the place where to insert the item
- **removeMin** and **min** take $O(1)$ time, since the smallest key is at the beginning

정렬된 배열, 정렬이 안된 배열로 구현하면 성능 차이가 존재한다.